

Learning-Augmented Page Replacement Under a Workload Shift

Student Name — Student ID
CSE-307 Operating Systems — Section B

Abstract

This paper evaluates FIFO, LRU, Belady's optimal replacement, and a lightweight decision-tree eviction heuristic on three independently seeded synthetic traces. Each trace changes at its midpoint from locality-heavy references to broad random access. We report means and sample standard deviations, examining how working-set changes affect faults and whether simple history features respond to the shift. Optimal provides an offline lower bound, not an implementable online policy.

1. Problem framing

Virtual memory allows a process to use a logical address space larger than physical memory, but a reference to a non-resident page causes a page fault. When a frame is full, the replacement policy chooses a victim. FIFO is simple but ignores reuse, while LRU uses the recent past as a proxy for near-future reuse. Belady's optimal policy chooses the page whose next use is farthest away and is useful as an offline comparison [1,2]. The course brief asks whether a lightweight learned layer can respond when the workload changes [4].

2. Implementation and learned component

The Python program implements all four policies. The learned policy uses a depth-4 decision tree. For each resident candidate, its features are age since last reference, frequency in the previous 64 references, and whether it was touched in the previous eight references. Training uses an independent seeded trace. Labels identify the candidate with the farthest next use in a 96-reference horizon. During evaluation, only current/history features are supplied. This is an interpretable heuristic, but the labels use future information during training, so it is not a fully online production predictor [3].

3. Experimental setup

Three evaluation traces use seeds 307, 308, and 309. Each contains 1,000 references. References 0–499 are locality-heavy: 82% of accesses select from a small hot set, with occasional sequential accesses. At reference 500, the distribution shifts to uniform random references over 32 pages. Each policy is tested with 4, 8, and 12 frames; the main comparison uses 8 frames. Each trace is replayed unchanged for all policies. The learned tree is trained once on an independent trace and held fixed. Per-seed traces and measurements are stored in results/.

4. Results and analysis

Table 1 reports the mean and sample standard deviation of page faults across three runs (500 accesses per phase). Optimal has the fewest faults because it sees the future. LRU performs better than FIFO in the locality phase, where reuse helps recency. Following the shift, every policy incurs substantially more faults as the request distribution broadens beyond the small hot set. The learned heuristic has the lowest locality-phase mean among the implementable policies, but its shifted-phase hit ratio is lower than those of FIFO and LRU. The fixed model's training labels and feature patterns do not generalize equally well to the shifted distribution, illustrating that learned heuristics can be sensitive to distribution mismatch.

Table 1. Eight-frame results from the seeded run.

Policy	Local faults mean $\pm$ SD	Local hit	Shift faults mean $\pm$ SD	Shift hit
FIFO	138.3 $\pm$ 8.1	0.723	375.0 $\pm$ 9.6	0.250
LRU	110.0 $\pm$ 6.6	0.780	372.7 $\pm$ 16.2	0.255
Optimal	57.3 $\pm$ 1.5	0.885	231.7 $\pm$ 5.9	0.537
Learned	79.0 $\pm$ 2.6	0.842	383.7 $\pm$ 11.6	0.233

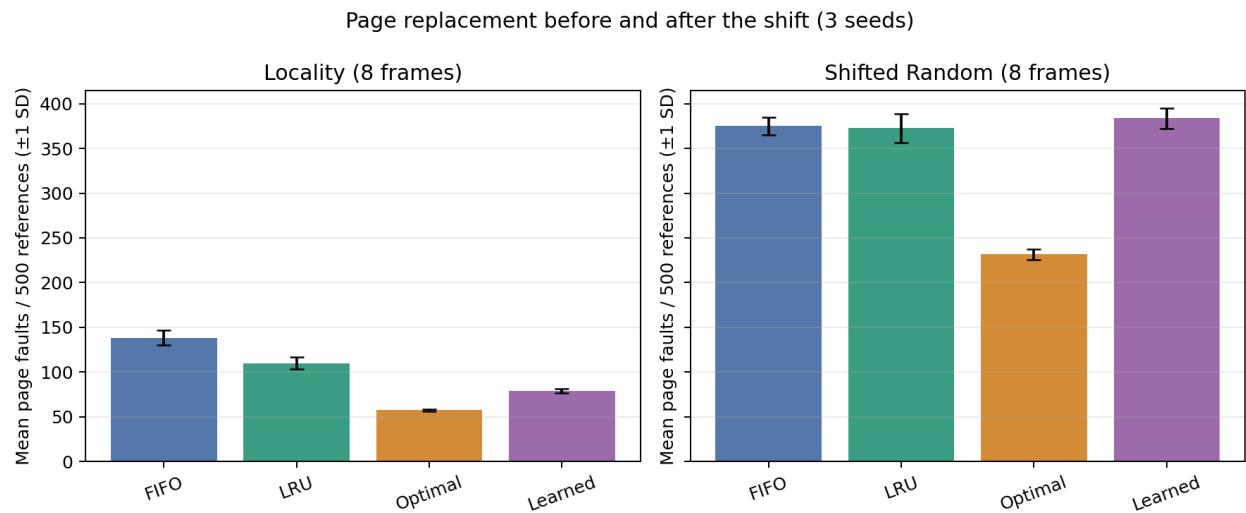


Figure 1. Fault counts before and after the distribution shift for eight frames.

5. Conclusion

Across the three generated seeds, the broad random phase consistently produces many more faults than the locality phase. The learned policy is strongest during locality for the tested 8-frame setup but does not retain that advantage after the workload shift. These synthetic results illustrate a limitation of using a fixed learned decision rule across changing distributions. Additional seeds, online labels, and real traces would be needed before making claims about deployment.

References

- [1] L. A. Belady, 'A study of replacement algorithms for a virtual-storage computer,' IBM Systems Journal, 1966.
- [2] R. L. Mattson et al., 'Evaluation techniques for storage hierarchies,' IBM Systems Journal, 1970.
- [3] scikit-learn developers, DecisionTreeClassifier documentation, 2025.
- [4] CSE-307 course staff, Term Paper Brief — Section B, 2026.